# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

**"JnanaSangama", Belgaum -590014, Karnataka.**

**LAB REPORT**

**on**

# OPERATING SYSTEMS

(23CS4PCOPS)

***Submitted by***

**Amogh Kamath Udyavara(1BF24CS044)**

***in partial fulfillment for the award of the degree of***

**BACHELOR OF ENGINEERING**

***in***

**COMPUTER SCIENCE AND ENGINEERING**

**B.M.S. COLLEGE OF ENGINEERING**

**(Autonomous Institution under VTU)**

**BENGALURU-560019**

**Feb-2026 to June-2026**

**B. M. S. College of Engineering,**

**Bull Temple Road, Bangalore 560019**

(Affiliated To Visvesvaraya Technological University, Belgaum)

## Department of Computer Science and Engineering

## CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by **Amogh Kamath Udyavara(1BF24CS044),** who is bonafide student of **B. M. S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2024. The Lab report has been approved as it satisfies the academic requirements in respect of a **OPERATING SYSTEMS - (23CS4PCOPS)** work prescribed for the said degree.


**Sandhya A. Kulkarni**
Assistant Professor
Department of CSE
BMSCE,Bengaluru

**Dr. Kavitha Sooda**
Professor and Head
Department of CSE
BMSCE, Bengaluru

# Index

**Course Outcome**

| | |
|---|---|
| CO1 | Apply the different concepts and functionalities of Operating System |
| CO2 | Analyze various Operating system strategies and techniques |
| CO3 | Demonstrate the different functionalities of Operating System |
| CO4 | Conduct practical experiments to implement the functionalities of Operating system |

**Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.**

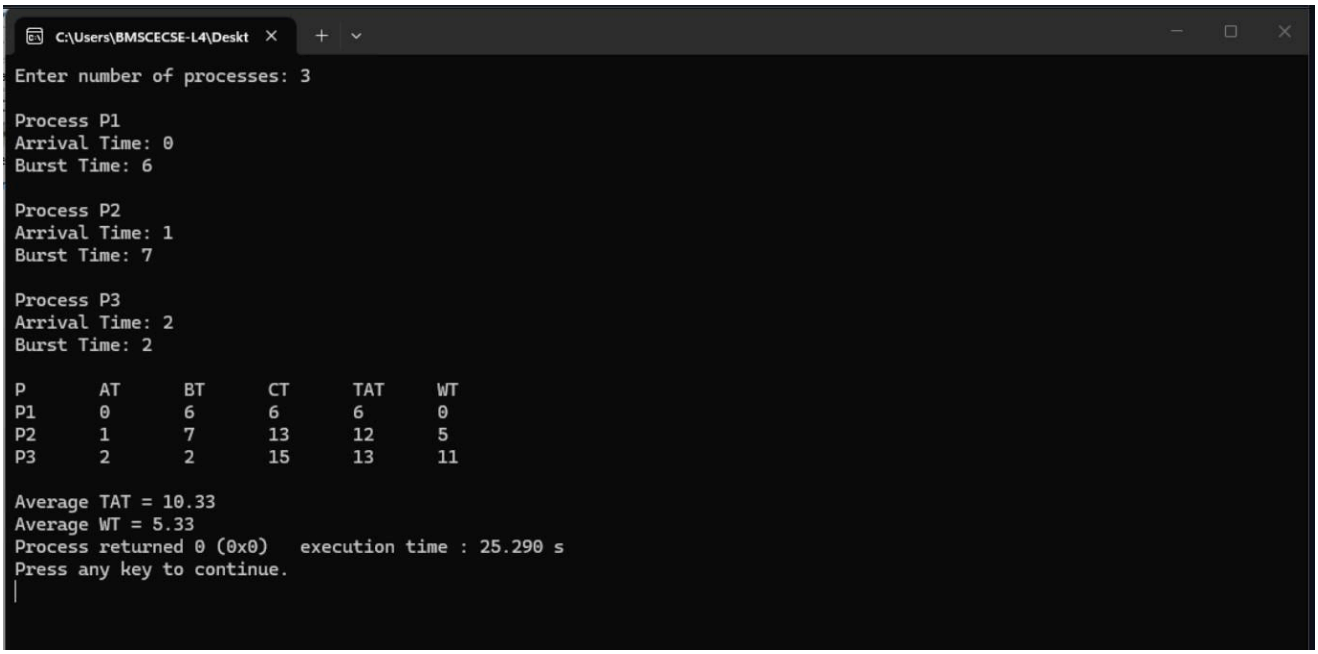
## a) FCFS

```
#include <stdio.h>

int main()
{
    int n, i;

    int at[20];   int bt[20];
    int ct[20];   int wt[20];   int
    tat[20];

    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nProcess P%d\n", i + 1);

        printf("Arrival Time : ");
        scanf("%d", &at[i]);

        printf("Burst Time   :
        "); scanf("%d", &bt[i]);
    }

    ct[0] = at[0] + bt[0];

    for(i = 1; i < n; i++)
    {
        if(ct[i - 1] < at[i])
            ct[i] = at[i] + bt[i];
```

```
        else
            ct[i] = ct[i - 1] + bt[i];
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];

        wt[i] = tat[i] - bt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
    }


printf("\n                                                                    \n");
    printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
    printf("                                                                    \n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1,
               at[i],
               bt[i],
               ct[i],
               tat[i],
               wt[i]);
    }

    printf("\nAverage Waiting Time = %.2f",
           avg_wt / n);

    printf("\nAverage Turnaround Time = %.2f\n",
           avg_tat / n);

    printf("\nGANTT CHART\n\n");

    for(i = 0; i < n; i++)
```

```
        printf("|  P%d  ", i + 1);
    printf("|\n");
    printf("%d", at[0]);
    for(i = 0; i < n; i++)
        printf("      %d", ct[i]);

    printf("\n");

    return 0;
}
```

## OUTPUT-FCFS



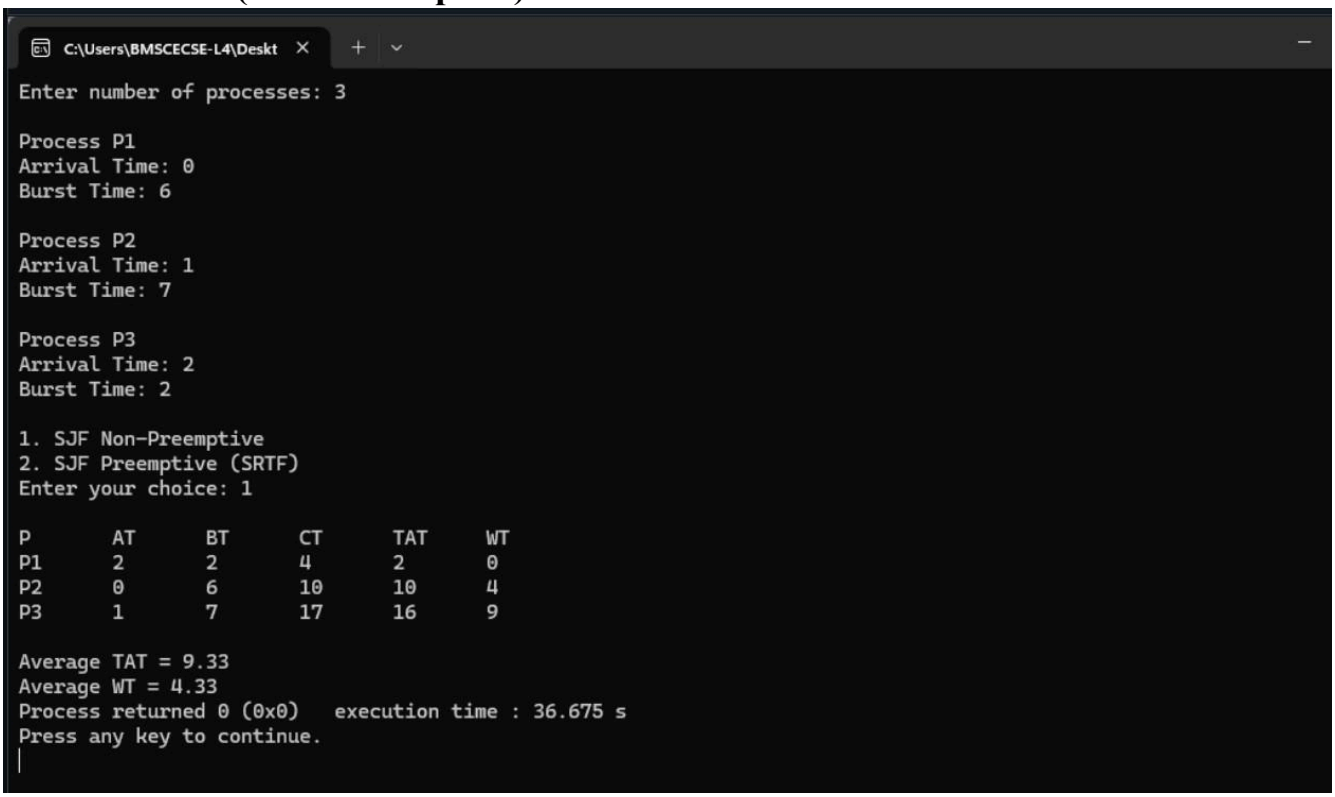
## b) SJF (Non-Preemptive)

```
#include <stdio.h>

int main()
{
    int n;
    int at[10];
    int bt[10];
    int ct[10];
    int tat[10];
    int wt[10];
    int done[10] = {0};
    int current_time = 0;
    int completed = 0;

    int min;
    int index;

    float avg_wt = 0;
    float avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
    {
        printf("\nEnter Arrival Time of P%d: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter Burst Time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    while(completed != n)
    {
                        min = 9999;
```

```
    index = -1;

    for(int i = 0; i < n; i++)
    {
        if(at[i] <= current_time &&
           done[i] == 0 &&
           bt[i] < min)
        {
            min = bt[i];

            index = i;
        }
    }

    if(index == -1)
    {
        current_time++;
    }
    else
    {
        current_time += bt[index];

        ct[index] = current_time;

        done[index] = 1;

        completed++;
    }
}

// Calculate TAT and WT
for(int i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    avg_tat += tat[i];
```

```
        avg_wt += wt[i];
    }
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1,
               at[i],
               bt[i],
               ct[i],
               tat[i],
               wt[i]);
    }

    printf("\nAverage TAT = %.2f", avg_tat / n);

    printf("\nAverage WT = %.2f", avg_wt / n);

    return 0;
}
```

## OUTPUT- SJF( Non Preemptive)




## SJF(Preemptive)- SRTF

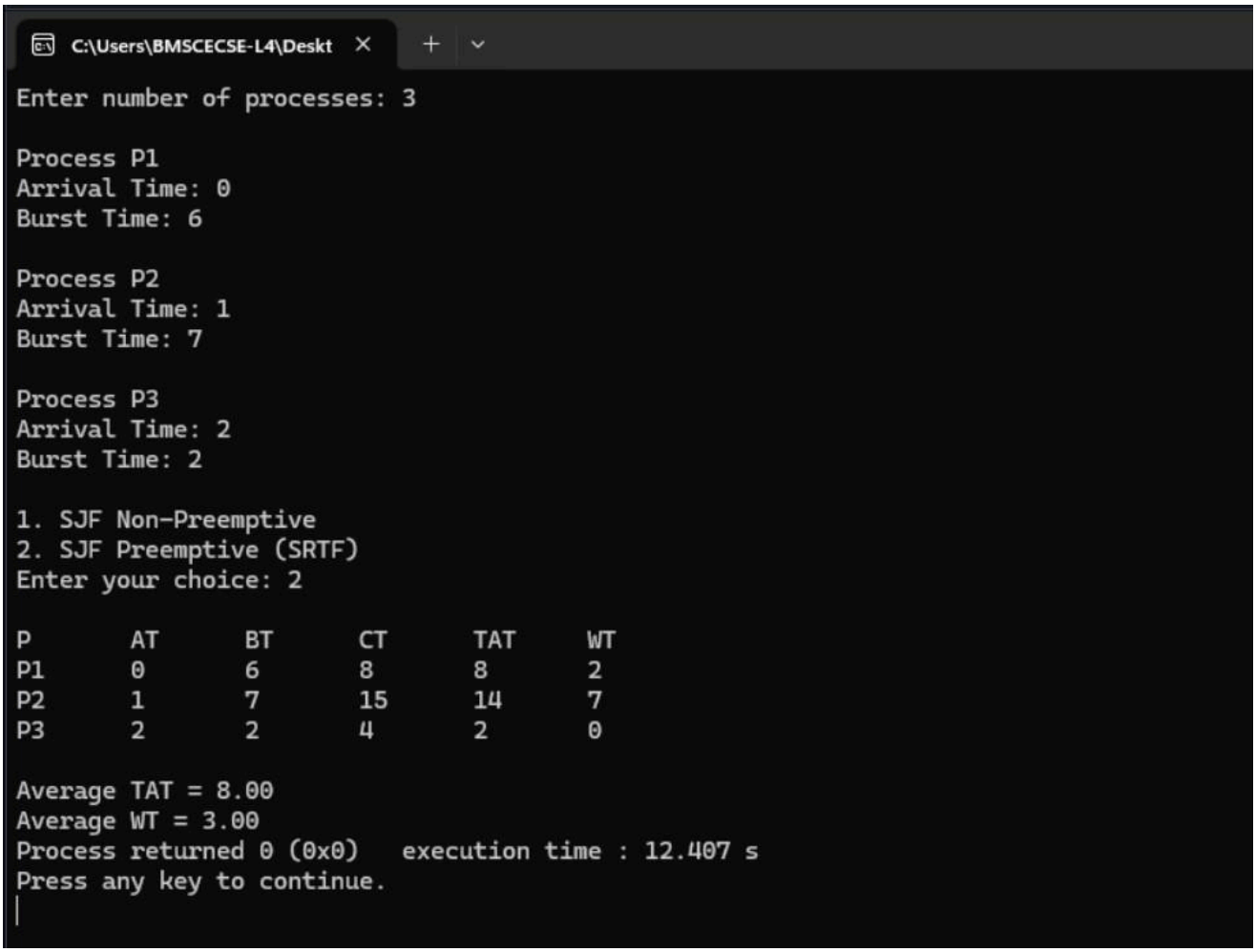
```
#include <stdio.h>

int main()
{
    int n;

    int at[10];
    int bt[10];
    int rt[10];
    int ct[10];
    int tat[10];
    int wt[10];

    int completed = 0;

    int current_time = 0;
    int min;
    int index;
```

```
float avg_wt = 0;
float avg_tat = 0;

printf("Enter number of processes: ");
scanf("%d", &n);

// Input Arrival Time and Burst Time
for(int i = 0; i < n; i++)
{
    printf("\nEnter Arrival Time of P%d: ", i + 1);
    scanf("%d", &at[i]);

    printf("Enter Burst Time of P%d: ", i + 1);
    scanf("%d", &bt[i]);

    rt[i] = bt[i];      // Initially remaining time = burst time
}

// Continue until all processes finish
while(completed != n)
{
    min = 9999;

    index = -1;

    // Find process with shortest remaining time
    for(int i = 0; i < n; i++)
    {
        if(at[i] <= current_time &&
           rt[i] > 0 &&
           rt[i] < min)
        {
            min = rt[i];

            index = i;
        }
    }

    // No process available
    if(index == -1)
```

```
    {
        current_time++;
    }
    else
    {
        rt[index]--;        // Execute for 1 unit

        current_time++;     // Increase CPU time

        // Process finished
        if(rt[index] == 0)
        {
            ct[index] = current_time;

            completed++;
        }
    }
}

// Calculate TAT and WT
for(int i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];

    wt[i] = tat[i] - bt[i];

    avg_tat += tat[i];

    avg_wt += wt[i];
}
printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
for(int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           i + 1,
           at[i],
           bt[i],
           ct[i],
           tat[i],
```

```
            wt[i]);
    }

    printf("\nAverage TAT = %.2f", avg_tat / n);

    printf("\nAverage WT = %.2f", avg_wt / n);

    return 0;
}
```

## OUTPUT- SJF(Preemptive)- SRTF



## c) Priority (Non-Preemptive)

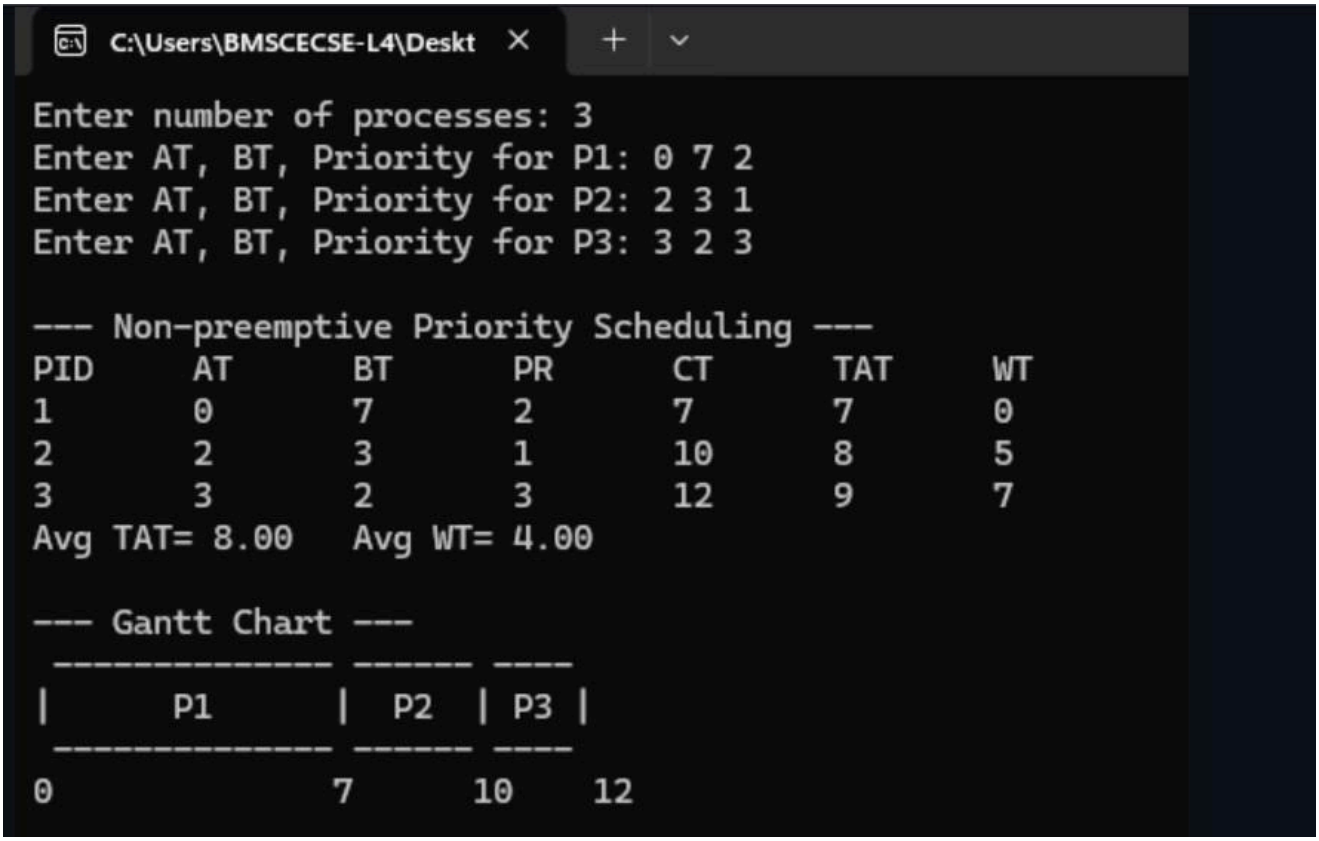
```
#include <stdio.h>

int main()
{
    int n;                          // Number of processes

    int at[10];                     // Arrival Time
    int bt[10];                     // Burst Time
    int pr[10];                     // Priority

    int ct[10];                     // Completion Time
    int tat[10];                    // Turnaround Time
    int wt[10];                     // Waiting Time

    int done[10] = {0};             // Process completion status

    int current_time = 0;           // CPU time
    int completed = 0;              // Completed processes count

    int min_priority;               // Highest priority value
    int index;                      // Selected process

    float avg_wt = 0;
    float avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    // Input data
    for(int
    i=0;i<n;i++)
    {
        printf("\nEnter Arrival Time of P%d: ",i+1);
        scanf("%d",&at[i]);

        printf("Enter Burst Time of P%d: ",i+1);
        scanf("%d",&bt[i]);

        printf("Enter Priority of P%d: ",i+1);
```

```
scanf("%d",&pr[i]);
```

```
}

// Continue until all processes finish
while(completed != n)
{
    min_priority = 9999;    // Assume very large priority
    index = -1;

    // Find highest priority process
    for(int i=0;i<n;i++)
    {
        if(at[i] <= current_time &&
           done[i] == 0 &&
           pr[i] < min_priority)
        {
            min_priority = pr[i];

            index = i;
        }
    }

    // No process available
    if(index == -1)
    {
        current_time++;
    }
    else
    {
        current_time += bt[index];   // Execute completely

        ct[index] = current_time;    // Store completion

        time done[index] = 1;        // Mark completed

        completed++;
    }
}

// Calculate TAT and WT
for(int i=0;i<n;i++)
{
```

```
        tat[i] = ct[i] - at[i];

        wt[i] = tat[i] - bt[i];

        avg_tat += tat[i];

        avg_wt += wt[i];
    }
    printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n"
               , i+1,
               at[i],
               bt[i],
               pr[i],
               ct[i],
               tat[i],
               wt[i]);
    }

    printf("\nAverage TAT = %.2f",avg_tat/n);

    printf("\nAverage WT = %.2f",avg_wt/n);

    return 0;
}
```

**OUTPUT-Priority (Non-Preemptive)**



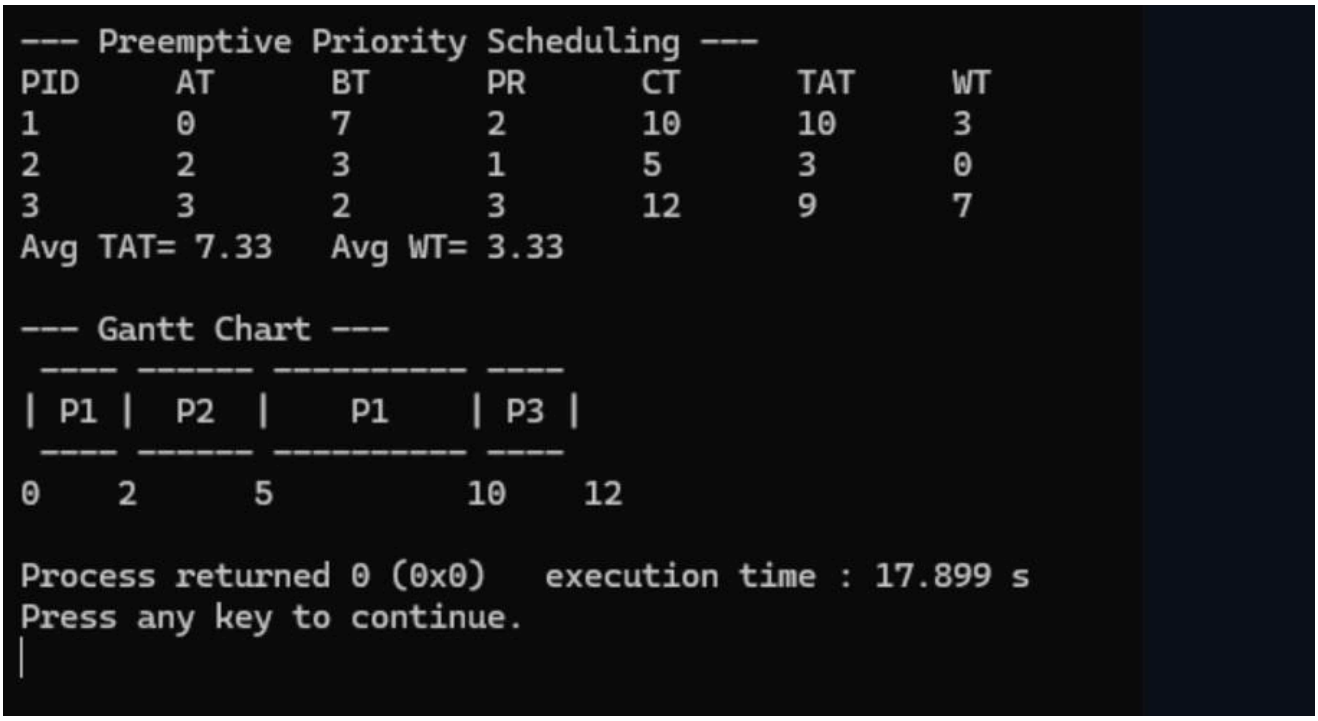
## Priority (Preemptive)

```
#include <stdio.h>

int main()
{
    int n;                          // Number of processes

    int at[10];                     // Arrival Time
    int bt[10];                     // Burst Time
    int rt[10];                     // Remaining Time
    int pr[10];                     // Priority

    int ct[10];                     // Completion Time
    int tat[10];                    // Turnaround Time
    int wt[10];                     // Waiting Time

    int current_time = 0;           // CPU clock
    int completed = 0;              // Completed processes

    int min_priority;               // Highest priority
    int index;                      // Selected process

    float avg_wt = 0;
    float avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    // Input
    for(int i=0;i<n;i++)
    {
        printf("\nEnter Arrival Time of P%d: ",i+1);
        scanf("%d",&at[i]);

        printf("Enter Burst Time of P%d: ",i+1);
        scanf("%d",&bt[i]);

        printf("Enter Priority of P%d: ",i+1);
        scanf("%d",&pr[i]);

        rt[i] = bt[i];      // Remaining time initially equals BT
```

```
}

while(completed != n)
{
    min_priority = 9999;

    index = -1;

    // Find highest priority process
    for(int i=0;i<n;i++)
    {
        if(at[i] <= current_time &&
           rt[i] > 0 &&
           pr[i] < min_priority)
        {
            min_priority = pr[i];

            index = i;
        }
    }

    // No process available
    if(index == -1)
    {
        current_time++;
    }
    else
    {
        rt[index]--;        // Execute for one

        unit current_time++;

        // Process finished
        if(rt[index] == 0)
        {
            ct[index] = current_time;

            completed++;
        }
    }
}
```

```
    // Calculate TAT and WT
    for(int i=0;i<n;i++)
    {
        tat[i] = ct[i] - at[i];

        wt[i] = tat[i] - bt[i];

        avg_tat += tat[i];

        avg_wt += wt[i];
    }
    printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n"
               , i+1,
               at[i],
               bt[i],
               pr[i],
               ct[i],
               tat[i],
               wt[i]);
    }

    printf("\nAverage TAT = %.2f",avg_tat/n);

    printf("\nAverage WT = %.2f",avg_wt/n);

    return 0;
}
```

**OUTPUT-Priority (Preemptive)**



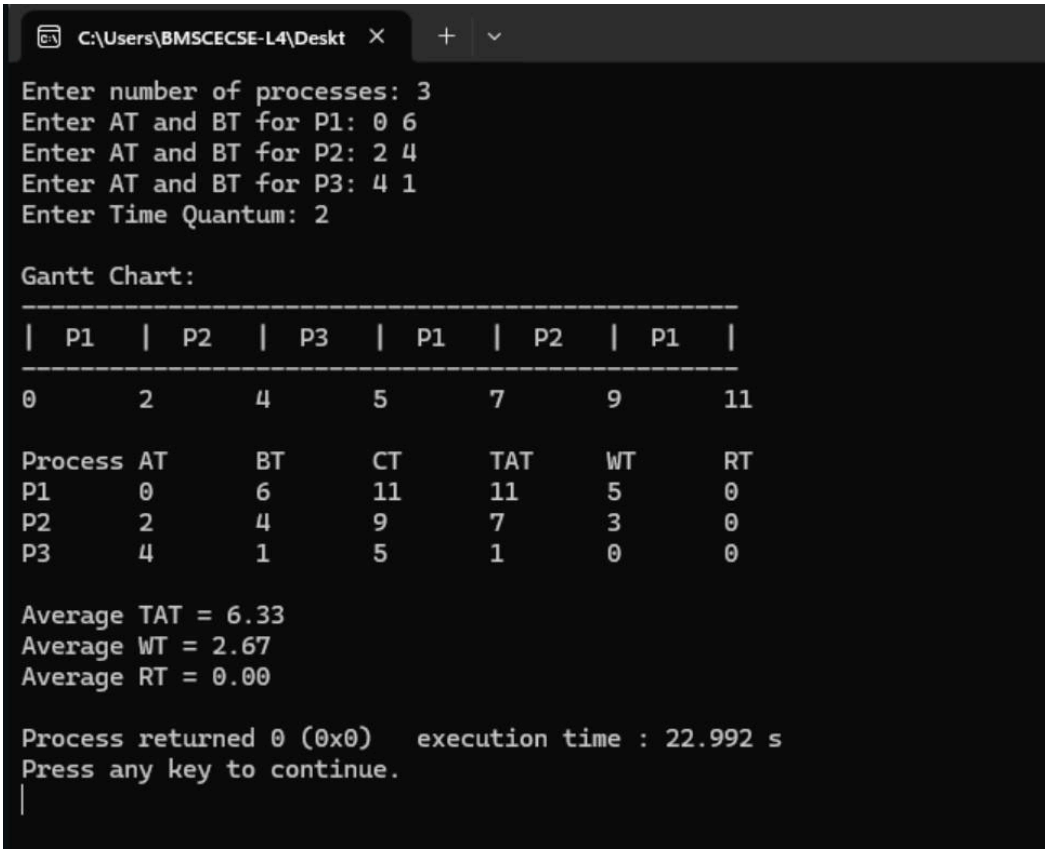
## d) Round Robin ( Experiment with different Time Quantum)

```
#include <stdio.h>

int main()
{
    int n;                          // Number of processes

    int at[10];                     // Arrival Time
    int bt[10];                     // Burst Time
    int rt[10];                     // Remaining Time

    int ct[10];                     // Completion Time
    int tat[10];                    // Turnaround Time
    int wt[10];                     // Waiting Time

    int tq;                         // Time Quantum

    int current_time = 0;           // CPU clock
    int completed = 0;              // Completed processes count

    float avg_wt = 0;
    float avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    // Input Arrival Time and Burst Time
    for(int i=0;i<n;i++)
    {
        printf("\nEnter Arrival Time of P%d: ",i+1);
        scanf("%d",&at[i]);

        printf("Enter Burst Time of P%d: ",i+1);
        scanf("%d",&bt[i]);

        rt[i] = bt[i];      // Initially Remaining Time = Burst Time
    }

    printf("\nEnter Time Quantum: ");
    scanf("%d",&tq);
```

```
// Continue until all processes complete
while(completed != n)
{
    int found = 0;      // Checks whether any process executed

    for(int i=0;i<n;i++)
    {
        // Process must have arrived and not completed
        if(at[i] <= current_time && rt[i] > 0)
        {
            found = 1;

            // Process can finish within quantum
            if(rt[i] <= tq)
            {
                current_time += rt[i];

                rt[i] = 0;

                ct[i] = current_time;

                completed++;
            }
            else
            {
                current_time += tq;

                rt[i] -= tq;
            }
        }
    }

    // No process available at current time
    if(found == 0)
    {
        current_time++;
    }
}

// Calculate TAT and WT
for(int i=0;i<n;i++)
```

```
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        avg_tat += tat[i];
        avg_wt += wt[i];
    }
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               i+1,
               at[i],
               bt[i],
               ct[i],
               tat[i],
               wt[i]);
    }

    printf("\nAverage TAT = %.2f",avg_tat/n);

    printf("\nAverage WT = %.2f",avg_wt/n);

    return 0;
}
```

## OUTPUT-Round Robin



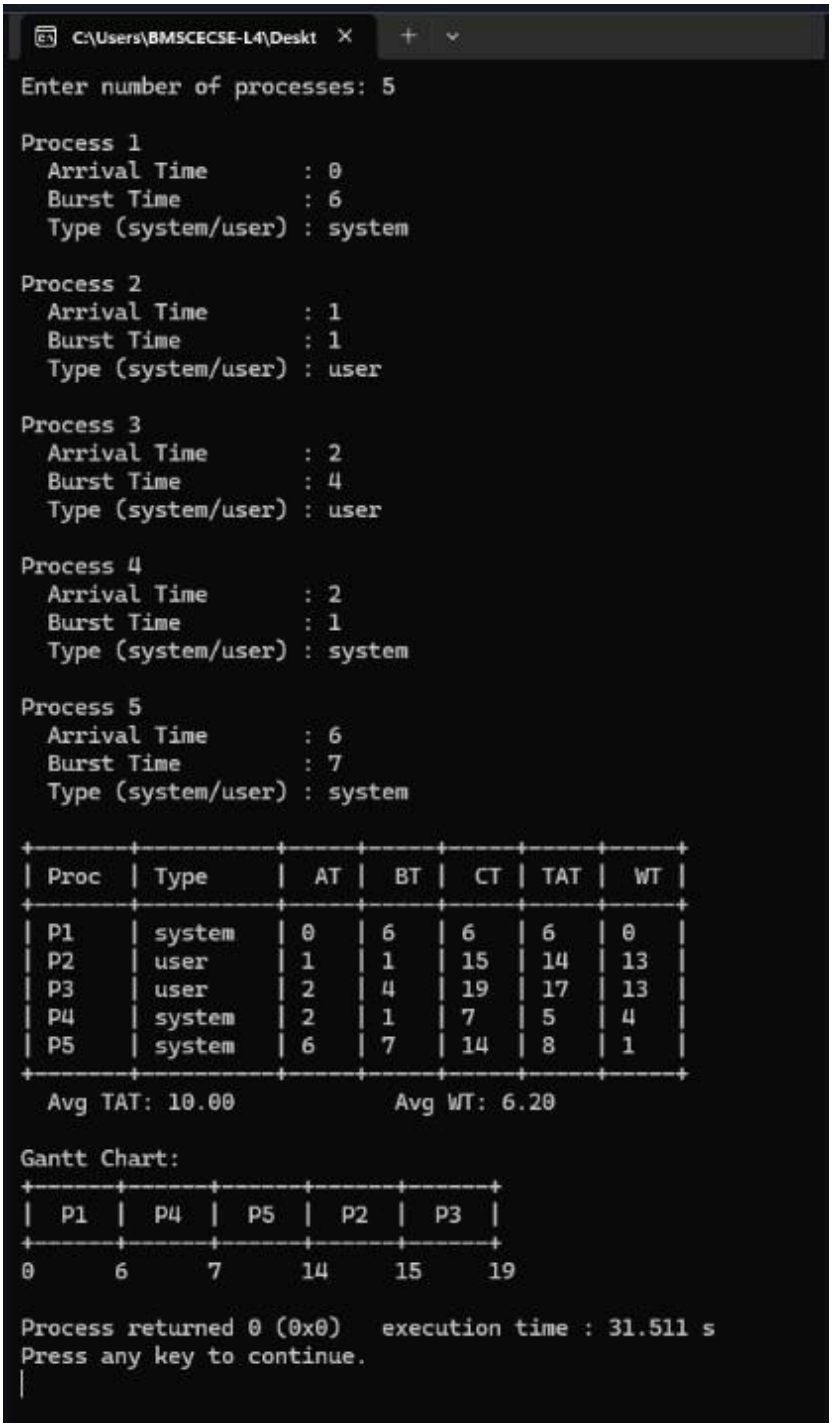
**Q2) Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.**

```
#include <stdio.h>

int main()

{
    int n;

    int at[10];      // Arrival
    Time int bt[10]; // Burst Time
    int type[10];    // 1-System, 2-User

    int ct[10];
    int tat[10];
    int wt[10];

    int current_time = 0;

    float avg_wt = 0;
    float avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(int i=0;i<n;i++)
    {
        printf("\nArrival Time of P%d: ",i+1);
        scanf("%d",&at[i]);

        printf("Burst Time of P%d: ",i+1);
        scanf("%d",&bt[i]);

        printf("Type (1-System,2-User): ");
        scanf("%d",&type[i]);
    }
```

```
// Execute System Processes First
for(int i=0;i<n;i++)
{
    if(type[i] == 1)
    {
        if(current_time < at[i])
            current_time = at[i];

        current_time += bt[i];

        ct[i] = current_time;
    }
}

// Execute User Processes Next
for(int i=0;i<n;i++)
{
    if(type[i] == 2)
    {
        if(current_time < at[i])
            current_time = at[i];

        current_time += bt[i];

        ct[i] = current_time;
    }
}

for(int i=0;i<n;i++)
{
    tat[i] = ct[i] - at[i];

    wt[i] = tat[i] - bt[i];

    avg_tat += tat[i];

    avg_wt += wt[i];
}

printf("\nP\tAT\tBT\tTYPE\tCT\tTAT\tWT\n");
```

```
    for(int i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n"
               , i+1,
               at[i],
               bt[i],
               type[i],
               ct[i],
               tat[i],
               wt[i]);
    }

    printf("\nAverage TAT = %.2f",avg_tat/n);

    printf("\nAverage WT = %.2f",avg_wt/n);

    return 0;
}
```

## OUTPUT- MULTILEVEL



## Q3)Write a C program to simulate Real-Time CPU Scheduling algorithms:

## a) Rate- Monotonic

```
#include <stdio.h>
#include <math.h>

int main()
{
    int n;                          // Number of tasks

    int task[10];                   // Task IDs
    int bt[10];                     // Execution Time (Burst Time)
    int period[10];                 // Period of tasks

    int ct[10];                     // Completion Time
    int tat[10];                    // Turnaround Time
    int wt[10];                     // Waiting Time

    int temp;                       // Temporary variable for swapping

    float utilization = 0;          // CPU Utilization
    float bound;                    // RMS Bound

    float avg_wt = 0;               // Average Waiting Time
    float avg_tat = 0;              // Average Turnaround Time


    printf("Enter number of tasks: ");
    scanf("%d", &n);

    // Input task details
    for(int i = 0; i < n; i++)
    {
        task[i] = i + 1;            // Store task number

        printf("\nEnter Execution Time of T%d: ", i + 1);
        scanf("%d", &bt[i]);

        printf("Enter Period of T%d: ", i + 1);
        scanf("%d", &period[i]);

        // Add contribution to CPU utilization
```

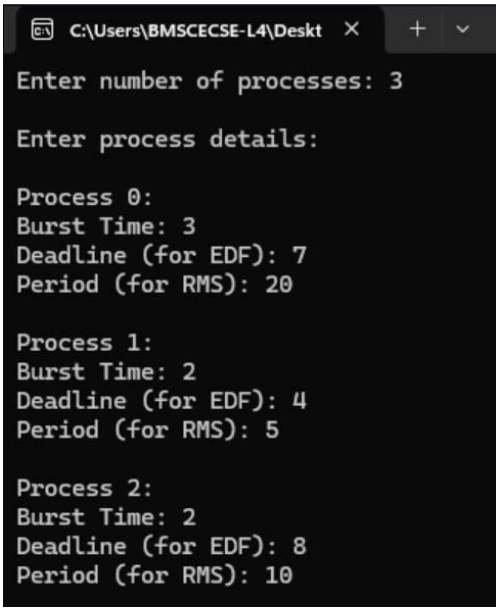
```
    utilization += (float)bt[i] / period[i];
}

// Sort tasks according to period
// Smaller period => Higher priority
for(int i = 0; i < n - 1; i++)
{
    for(int j = i + 1; j < n; j++)
    {
        if(period[i] > period[j])
        {
            // Swap period
            temp = period[i];
            period[i] = period[j];
            period[j] = temp;

            // Swap burst
            time temp =
            bt[i]; bt[i] =
            bt[j]; bt[j] =
            temp;

            // Swap task number
            temp = task[i];
            task[i] = task[j];
            task[j] = temp;
        }
    }
}

int time = 0;                   // Current CPU time

// Calculate WT, CT and TAT
for(int i = 0; i < n; i++)
{
    wt[i] = time;               // Waiting Time

    time += bt[i];              // Execute task

    ct[i] = time;               // Completion Time
```

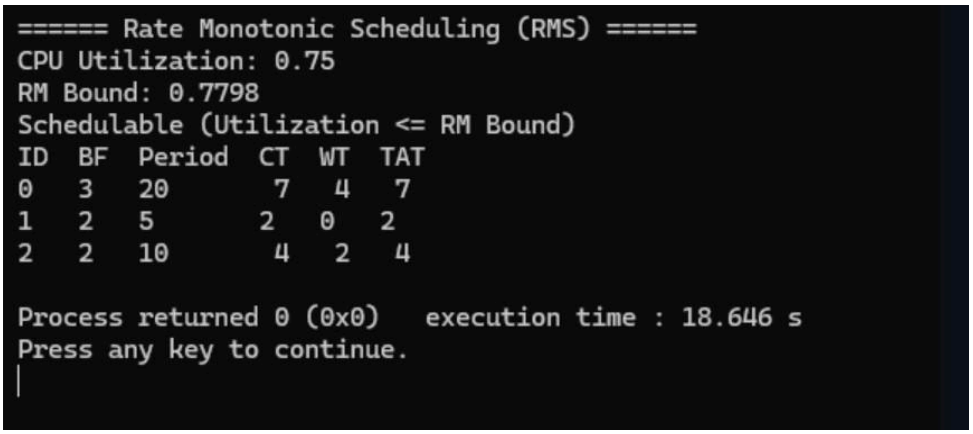
```
tat[i] = ct[i];              // Since Arrival Time = 0
```

```
    avg_wt += wt[i];            // Add WT for average

    avg_tat += tat[i];          // Add TAT for average
}

// Calculate RMS Bound
bound = n * (pow(2.0, 1.0 / n) - 1);

printf("\n\nRate Monotonic Scheduling\n");

printf("\nTask\tBT\tPeriod\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++)
{
    printf("T%d\t%d\t%d\t%d\t%d\t%d\n",
           task[i],
           bt[i],
           period[i],
           ct[i],
           tat[i],
           wt[i]);
}

printf("\nAverage Turnaround Time = %.2f",
       avg_tat / n);

printf("\nAverage Waiting Time = %.2f",
       avg_wt / n);

printf("\n\nCPU Utilization = %.4f",
       utilization);

printf("\nRMS Bound = %.4f",
       bound);

// Schedulability Test
if(utilization <= bound)
    printf("\nTask Set is Schedulable under RMS");
else
    printf("\nTask Set is NOT Guaranteed Schedulable under RMS");
```

```
    return 0;
}
```

## OUTPUT- Rate- Monotonic



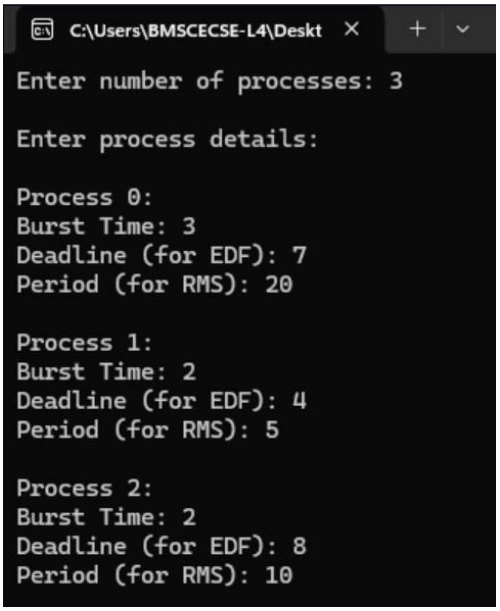
## b) Earliest-deadline First

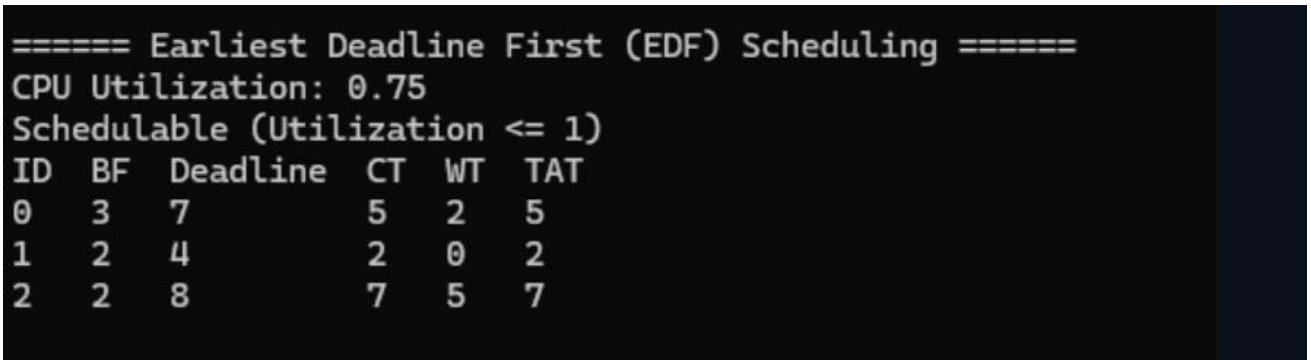
```
#include <stdio.h>

int main()
{
    int n;                          // Number of tasks

    int task[10];                   // Task IDs
    int bt[10];                     // Execution Time
    int deadline[10];               // Deadlines

    int ct[10];                     // Completion Time
    int tat[10];                    // Turnaround Time
    int wt[10];                     // Waiting Time

    int temp;                       // Temporary variable for swapping

    float utilization = 0;          // CPU Utilization

    float avg_wt = 0;               // Average Waiting Time
    float avg_tat = 0;              // Average Turnaround Time

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    // Input task details
    for(int i = 0; i < n; i++)
    {
        task[i] = i + 1;            // Store task number

        printf("\nEnter Execution Time of T%d: ", i + 1);
        scanf("%d", &bt[i]);

        printf("Enter Deadline of T%d: ", i + 1);
        scanf("%d", &deadline[i]);

        // Utilization contribution
        utilization += (float)bt[i] / deadline[i];
    }

    // Sort according to deadline
```

```
// Smaller deadline => Higher priority
for(int i = 0; i < n - 1; i++)
{
    for(int j = i + 1; j < n; j++)
    {
        if(deadline[i] > deadline[j])
        {
            // Swap deadline
            temp = deadline[i];
            deadline[i] = deadline[j];
            deadline[j] = temp;

            // Swap execution time
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;

            // Swap task number
            temp = task[i];
            task[i] = task[j];
            task[j] = temp;
        }
    }
}

int time = 0;                   // Current CPU time

// Calculate WT, CT and TAT
for(int i = 0; i < n; i++)
{
    wt[i] = time;               // Waiting Time

    time += bt[i];              // Execute task

    ct[i] = time;               // Completion Time

    tat[i] = ct[i];             // AT assumed

    0 avg_wt += wt[i];

    avg_tat += tat[i];
```

```
    }
    printf("\n\nEarliest Deadline First
    Scheduling\n");
    printf("\nTask\tBT\tDeadline\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++)
    {
        printf("T%d\t%d\t%d\t\t%d\t%d\t%d\n",
               task[i],
               bt[i],
               deadline[i],
               ct[i],
               tat[i],
               wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f",
           avg_tat / n);

    printf("\nAverage Waiting Time = %.2f",
           avg_wt / n);

    printf("\n\nCPU Utilization = %.4f",
           utilization);

    // EDF schedulability test
    if(utilization <= 1.0)
        printf("\nTask Set is Schedulable under EDF");
    else
        printf("\nTask Set is NOT Schedulable under EDF");

    return 0;
}
```

## OUTPUT-Earliest-deadline First



## Q4)Write a C program to simulate:

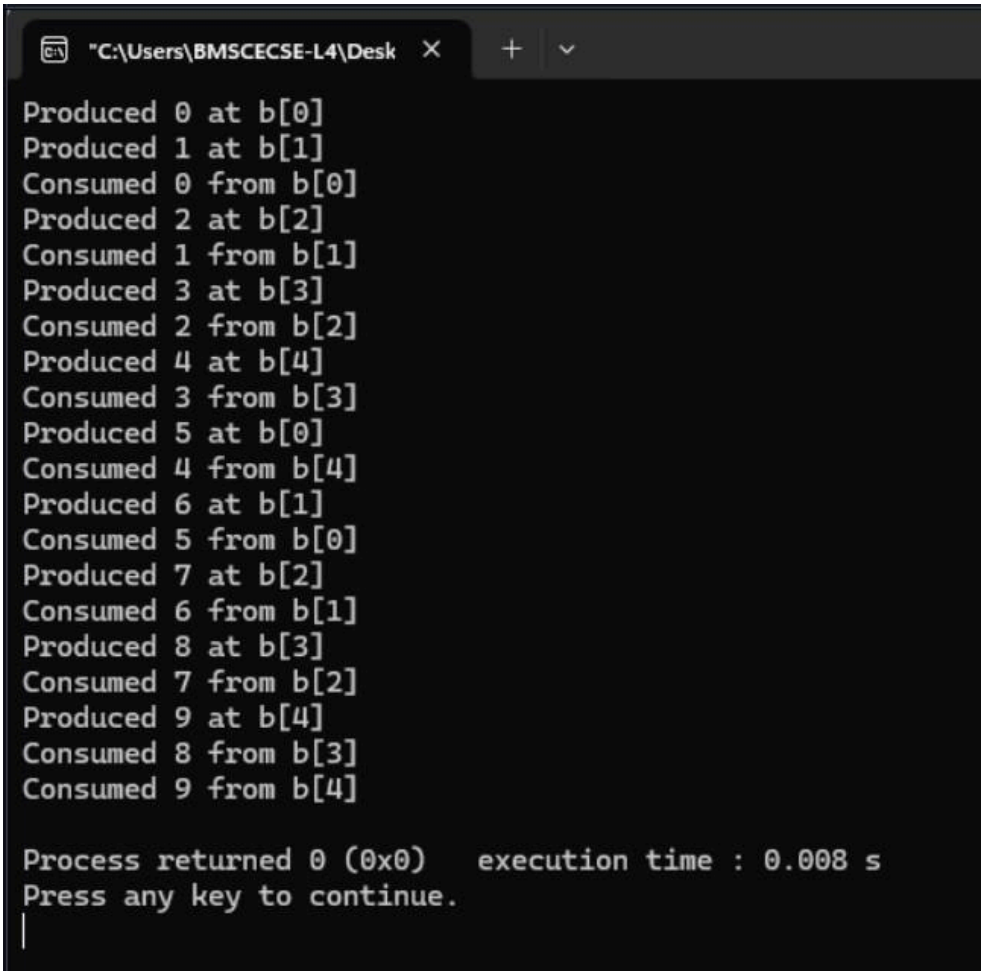
## a) Producer-Consumer problem using semaphores.

```
#include <stdio.h>              // For printf()

#define BUFFER_SIZE 5           // Size of buffer

// Shared buffer
int buffer[BUFFER_SIZE];

// Producer inserts at 'in'
int in = 0;

// Consumer removes from 'out'
int out = 0;

// Semaphore variables
int mutex = 1;                  // Mutual exclusion semaphore
int full = 0;                   // Number of filled
slots int empty = BUFFER_SIZE;  // Number of empty slots

// Wait operation (P operation)
// Decreases semaphore value
void wait(int *s)
{
    (*s)--;
}

// Signal operation (V operation)
// Increases semaphore value
void signal(int *s)
{
    (*s)++;
}

// Producer function
void producer(int item)
{
    // Check if buffer is full
    if(empty == 0)
    {
        printf("Buffer is Full\n");
```

```
        return;
    }

    // Decrease empty count
    wait(&empty);

    // Enter critical section
    wait(&mutex);

    // Insert item into buffer
    buffer[in] = item;

    printf("Produced: %d at Buffer[%d]\n",
           item, in);

    // Move to next buffer position
    // Circular queue implementation
    in = (in + 1) % BUFFER_SIZE;

    // Exit critical section
    signal(&mutex);

    // Increase full count
    signal(&full);
}

// Consumer function
void consumer()
{
    int item;

    // Check if buffer is empty
    if(full == 0)
    {
        printf("Buffer is Empty\n");
        return;
    }

    // Decrease full count
    wait(&full);
```

```
    // Enter critical section
    wait(&mutex);

    // Remove item from buffer
    item = buffer[out];

    printf("Consumed: %d from Buffer[%d]\n",
           item, out);

    // Move to next buffer position
    // Circular queue implementation
    out = (out + 1) % BUFFER_SIZE;

    // Exit critical section
    signal(&mutex);

    // Increase empty count
    signal(&empty);
}

int main()
{
    int item = 0;       // Item number

    // Simulate Producer and Consumer
    for(int i = 0; i < 15; i++)
    {
        // Produce an item
        producer(item++);

        // Consume after every alternate production
        if(i % 2 == 0)
        {
            consumer();
        }
    }

    return 0;
}
```

## OUTPUT-Producer-Consumer problem

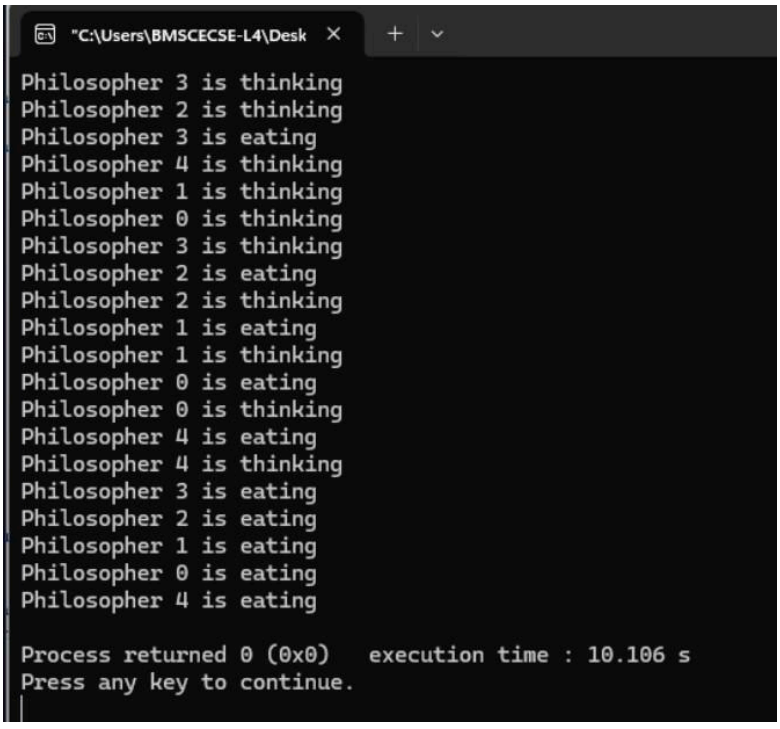



## b) Dining-Philosopher's problem

```
#include<stdio.h>
#define N 5
int chopstick[N] = {1, 1, 1, 1, 1};
int room = N - 1;

void wait(int *s){
   (*s)--;
}

void signal(int *s){
   (*s)++;
}

void philosopher(int i){
```

```
  printf("Phillosopher %d is thinking\n", i);
  wait(&room);
  wait(&chopstick[i]);
  printf("Philosopher %d picked up left fork %d\n", i, i);
  wait(&chopstick[(i+1)%N]);
  printf("Philosopher %d picked up right fork %d\n",i,
  (i+1)%N); printf("Philosopher %d is eating\n", i);
  signal(&chopstick[i]);
  signal(&chopstick[(i+1)%N]);
  signal(&room);
  printf("Philosopher %d is thinking.\n", i);

}

int main(){
  int i;
  for(int i = 0; i < N; i++){
    philosopher(i);
    printf("\n");
  }
  return 0;
}
```

## OUTPUT-Dining-Philosopher's problem



## Q5)Write a C program to simulate:

## a) Bankers' algorithm for the purpose of deadlock avoidance.

## Safety Algorithm

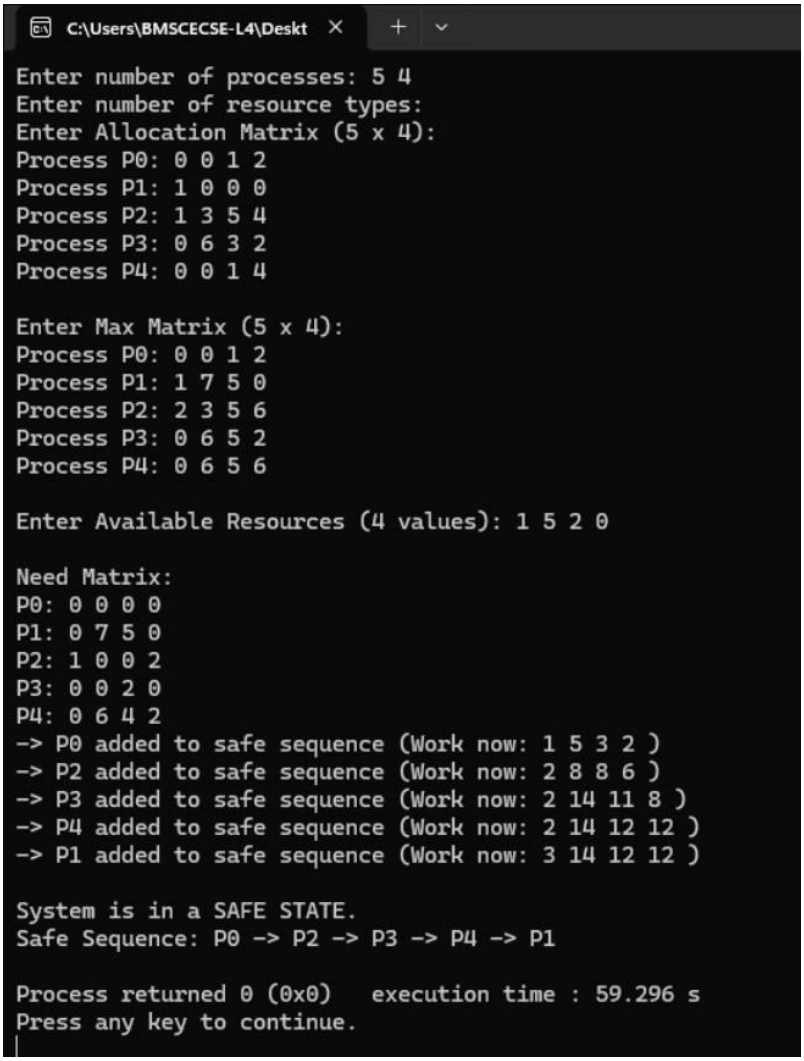
```
#include <stdio.h>

int main()
{
    int n, m;                          // n = processes, m = resources

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[10][10];                 // Allocation Matrix
    int max[10][10];                   // Maximum Matrix
    int need[10][10];                  // Need Matrix

    int avail[10];                     // Available Resources
    int work[10];                      // Working copy of Available

    int finish[10] = {0};              // 0 = not completed, 1 =

    completed int safe[10];            // Safe Sequence

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Input Maximum Matrix
    printf("\nEnter Maximum Matrix:\n");
```

```
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

// Input Available Resources
printf("\nEnter Available Resources:\n");

for(int j = 0; j < m; j++)
{
    scanf("%d", &avail[j]);

    // Copy Available into Work
    work[j] = avail[j];
}

// Calculate Need Matrix
// Need = Max - Allocation
printf("\nNeed Matrix:\n");

for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];

        printf("%d ", need[i][j]);
    }

    printf("\n");
}

int count = 0;                      // Number of completed processes

// Find Safe Sequence
while(count < n)
{
    int found = 0;                  // Indicates whether a process is found
```

```
for(int i = 0; i < n; i++)
{
    // Consider only unfinished processes
    if(finish[i] == 0)
    {
        int possible = 1;       // Assume process can execute

        // Check Need <= Work
        for(int j = 0; j < m; j++)
        {
            if(need[i][j] > work[j])
            {
                possible = 0;  // Process cannot
                execute break;
            }
        }

        // If process can execute
        if(possible)
        {
            // Release allocated resources
            for(int j = 0; j < m; j++)
            {
                work[j] += alloc[i][j];
            }

            // Add process to safe sequence
            safe[count] = i;

            // Mark process as completed
            finish[i] = 1;

            count++;

            found = 1;
        }
    }
}

// No process could execute
```

```
            if(found == 0)
            {
                break;
            }
        }

        // Check whether system is safe
        if(count == n)
        {
            printf("\nSystem is in Safe State\n");

            printf("Safe Sequence: ");

            for(int i = 0; i < n; i++)
            {
                printf("P%d ", safe[i]);
            }

            printf("\n");
        }
        else
        {
            printf("\nSystem is NOT in Safe State\n");
        }

        return 0;
}
```

## OUTPUT-Bankers' algorithm




## Resource Request Algorithm

```
#include <stdio.h>

int main()
{
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);
```

```
int alloc[10][10];
int max[10][10];
int need[10][10];

int avail[10];
int work[10];

int finish[10] = {0};
int safe[10];

// Allocation Matrix
printf("\nEnter Allocation Matrix:\n");

for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        scanf("%d", &alloc[i][j]);
    }
}

// Maximum Matrix
printf("\nEnter Maximum Matrix:\n");

for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

// Available Vector
printf("\nEnter Available Resources:\n");

for(int j = 0; j < m; j++)
{
    scanf("%d", &avail[j]);
}

// Need Matrix = Max - Allocation
```

```
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int p;                        // Process making
request int request[10];

printf("\nEnter process number: ");
scanf("%d", &p);

printf("Enter request vector:\n");

for(int j = 0; j < m; j++)
{
    scanf("%d", &request[j]);
}

// Check Request <= Need
for(int j = 0; j < m; j++)
{
    if(request[j] > need[p][j])
    {
        printf("\nError: Request exceeds Need\n");
        return 0;
    }
}

// Check Request <= Available
for(int j = 0; j < m; j++)
{
    if(request[j] > avail[j])
    {
        printf("\nProcess must wait\n");
        return 0;
    }
}
```

```
// Temporary Allocation
for(int j = 0; j < m; j++)
{
    avail[j] -= request[j];

    alloc[p][j] += request[j];

    need[p][j] -= request[j];
}

// Work = Available
for(int j = 0; j < m;
j++)
{
    work[j] = avail[j];
}

int count = 0;

// Safety Algorithm
while(count < n)
{
    int found = 0;

    for(int i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(int j = 0; j < m; j++)
            {
                if(need[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if(possible)
            {for(int j = 0; j < m; j++
```

```
{
                        work[j] += alloc[i][j];
                    }

                    safe[count] = i;

                    finish[i] = 1;

                    count++;

                    found = 1;
                }
            }
        }

        if(found == 0)
        {
            break;
        }
    }

    // Final Decision
    if(count == n)
    {
        printf("\nRequest can be granted\n");

        printf("Safe Sequence: ");

        for(int i = 0; i < n; i++)
        {
            printf("P%d ", safe[i]);
        }

        printf("\n");
    }
    else
    {
        printf("\nRequest cannot be granted\n");
    }

    return 0;
```

```
}
```

## b) Deadlock Detection

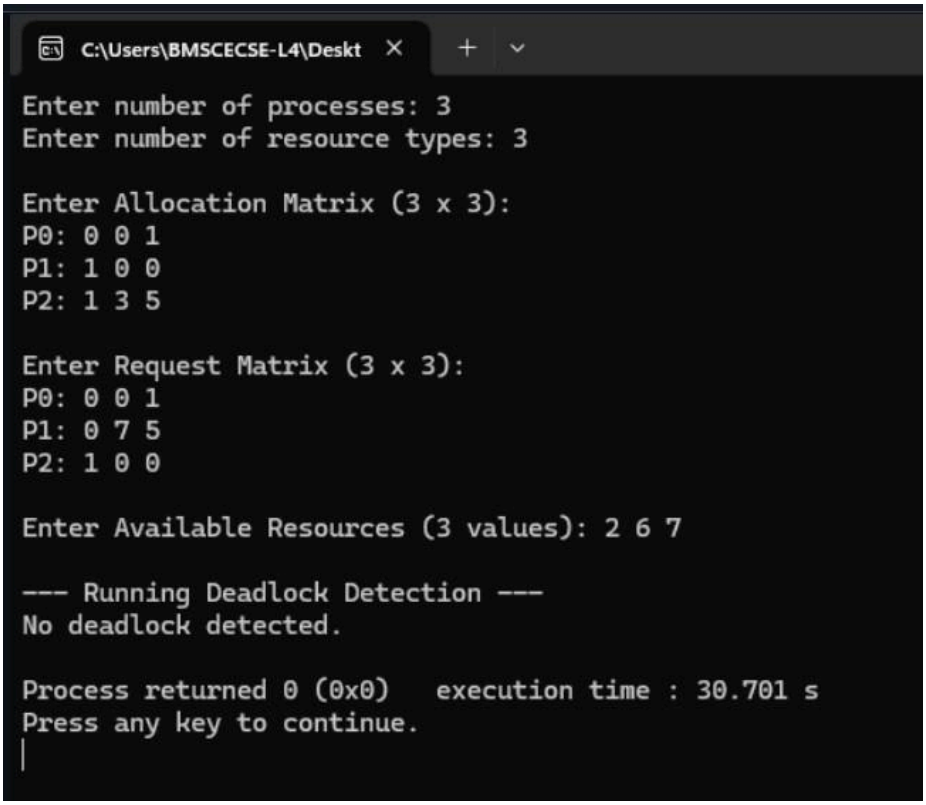
```
#include <stdio.h>

int main()
{
    int n, m;                          // n = processes, m = resources

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[10][10];                 // Allocation Matrix
    int request[10][10];               // Request Matrix

    int avail[10];                     // Available Vector
    int work[10];                      // Working copy of Available

    int finish[10] = {0};              // Finish Array

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Input Request Matrix
    printf("\nEnter Request Matrix:\n");

    for(int i = 0; i < n; i++)
    {
```

```
    for(int j = 0; j < m; j++)
    {
        scanf("%d", &request[i][j]);
    }
}

// Input Available Vector
printf("\nEnter Available Resources:\n");

for(int j = 0; j < m; j++)
{
    scanf("%d", &avail[j]);

    work[j] = avail[j];        // Copy Available into Work
}

int count = 0;                 // Number of completed processes

// Detection Algorithm
while(count < n)
{
    int found = 0;

    for(int i = 0; i < n; i++)
    {
        // Check only unfinished processes
        if(finish[i] == 0)
        {
            int possible = 1;

            // Check Request <= Work
            for(int j = 0; j < m; j++)
            {
                if(request[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            // Process can execute
```

```
            if(possible)
            {
                // Release allocated resources
                for(int j = 0; j < m; j++)
                {
                    work[j] += alloc[i][j];
                }

                finish[i] = 1;

                count++;

                found = 1;
            }
        }
    }

    // No process can execute
    if(found == 0)
    {
        break;
    }
}

// Check for deadlocked processes
if(count == n)
{
    printf("\nNo Deadlock Detected\n");
}
else
{
    printf("\nDeadlocked Processes: ");

    for(int i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            printf("P%d ", i);
        }
    }
```

```
        printf("\n");
    }

    return 0;
}
```

## OUTPUT- Deadlock Detection

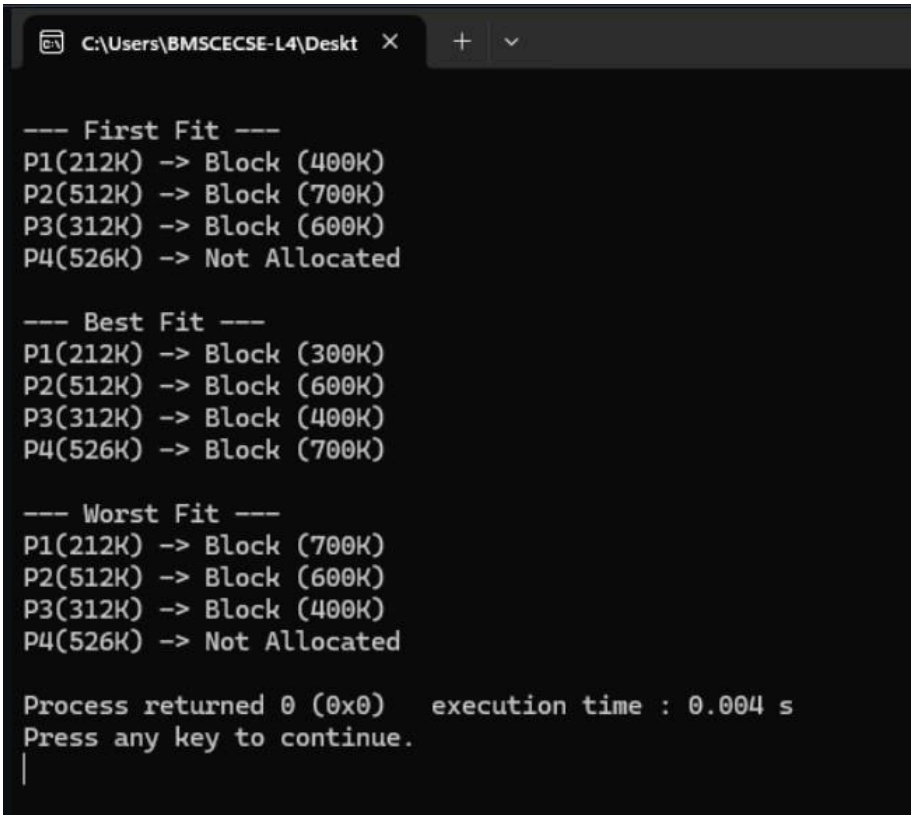


**Q6) Write a C program to simulate the following contiguous memory allocation techniques**
**a) Worst-fit**
**b) Best-fit**
**c) First-fit**

```
#include <stdio.h>

void firstFit(int b[], int m, int p[], int n) {
    printf("\n--- First Fit ---\n");
    int t[m];
    for(int i=0;i<m;i++) t[i]=b[i];
    for(int i=0;i<n;i++){
        int j;
        for(j=0;j<m;j++){
            if(t[j]>=p[i]){ t[j]=0; break; }  // mark block as used
        }
        if(j<m)
            printf("P%d(%dK) -> Block (%dK)\n", i+1, p[i],
        b[j]); else
            printf("P%d(%dK) -> Not Allocated\n", i+1, p[i]);
    }
}

void bestFit(int b[], int m, int p[], int n) {
    printf("\n--- Best Fit ---\n");
    int t[m];
    for(int i=0;i<m;i++) t[i]=b[i];
    for(int i=0;i<n;i++){
        int best=-1;
        for(int j=0;j<m;j++){
            if(t[j]>=p[i])
                if(best==-1 || t[j]<t[best]) best=j;
        }
        if(best!=-1){
            printf("P%d(%dK) -> Block (%dK)\n", i+1, p[i], b[best]);
            t[best]=0;  // mark block as used
        } else
            printf("P%d(%dK) -> Not Allocated\n", i+1, p[i]);
    }
```

```
}

void worstFit(int b[], int m, int p[], int n) {
    printf("\n--- Worst Fit ---\n");
    int t[m];
    for(int i=0;i<m;i++) t[i]=b[i];
    for(int i=0;i<n;i++){
        int worst=-1;
        for(int j=0;j<m;j++){
            if(t[j]>=p[i])
                if(worst==-1 || t[j]>t[worst]) worst=j;
        }
        if(worst!=-1){
            printf("P%d(%dK) -> Block (%dK)\n", i+1, p[i], b[worst]);
            t[worst]=0;  // mark block as used
        } else
            printf("P%d(%dK) -> Not Allocated\n", i+1, p[i]);
    }
}

int main() {
    int blocks[]    = {400, 700, 200, 300, 600};
    int processes[] = {212, 512, 312, 526};
    int m = 5, n = 4;

    firstFit(blocks, m, processes, n);
    bestFit(blocks, m, processes, n);
    worstFit(blocks, m, processes, n);

    return 0;
}
```

## OUTPUT


C:\Users\BMSCECSE-L4\Deskt
--- First Fit ---
P1(212K) -> Block (400K)
P2(512K) -> Block (700K)
P3(312K) -> Block (600K)
P4(526K) -> Not Allocated
--- Best Fit ---
P1(212K) -> Block (300K)
P2(512K) -> Block (600K)
P3(312K) -> Block (400K)
P4(526K) -> Block (700K)
--- Worst Fit ---
P1(212K) -> Block (700K)
P2(512K) -> Block (600K)
P3(312K) -> Block (400K)
P4(526K) -> Not Allocated
Process returned 0 (0x0) execution time : 0.004 s
Press any key to continue.

**Q7) Write a C program to simulate page replacement algorithms.**

**a) FIFO**

**b) LRU**

**c) Optimal**

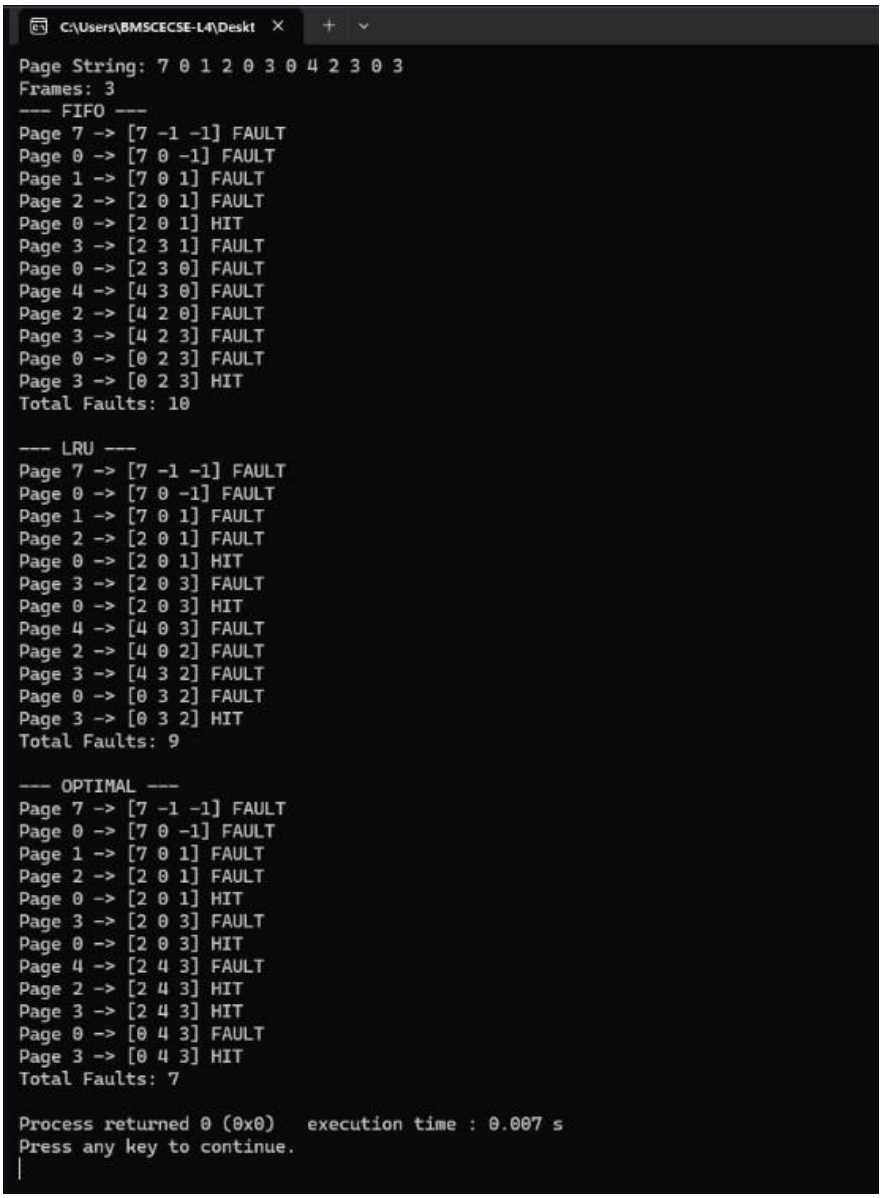
```
#include <stdio.h>
#include <string.h>

#define FRAMES 3
#define PAGES  12

int pages[PAGES] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3};

int inFrames(int frames[], int page) {
    for (int i = 0; i < FRAMES; i++)
        if (frames[i] == page) return 1;
    return 0;
}

void FIFO() {
    int frames[FRAMES] = {-1, -1, -1};
    int faults = 0, ptr = 0;

    printf("\n--- FIFO ---\n");
    for (int i = 0; i < PAGES; i++) {
        if (!inFrames(frames, pages[i])) {
            frames[ptr] = pages[i];
            ptr = (ptr + 1) % FRAMES;
            faults++;
            printf("Page %d -> [%d %d %d] FAULT\n", pages[i], frames[0],
frames[1], frames[2]);
        } else {
            printf("Page %d -> [%d %d %d] HIT\n", pages[i], frames[0],
frames[1], frames[2]);
        }
    }
    printf("Total Faults: %d\n", faults);
}

void LRU() {
```

```
    int frames[FRAMES] = {-1, -1, -1};
    int lastUsed[FRAMES] = {0};
    int faults = 0;

    printf("\n--- LRU ---\n");
    for (int i = 0; i < PAGES; i++) {
        if (!inFrames(frames, pages[i])) {
            // Find LRU slot (prefer empty slot first)
            int replace = 0;
            for (int j = 0; j < FRAMES; j++) {
                if (frames[j] == -1) { replace = j; break; }
                if (lastUsed[j] < lastUsed[replace]) replace = j;
            }
            frames[replace] = pages[i];
            lastUsed[replace] = i;
            faults++;
            printf("Page %d -> [%d %d %d] FAULT\n", pages[i], frames[0],
frames[1], frames[2]);
        } else {
            for (int j = 0; j < FRAMES; j++)
                if (frames[j] == pages[i]) lastUsed[j] = i;
            printf("Page %d -> [%d %d %d] HIT\n", pages[i], frames[0],
frames[1], frames[2]);
        }
    }
    printf("Total Faults: %d\n", faults);
}

void Optimal() {
    int frames[FRAMES] = {-1, -1, -1};
    int faults = 0, filled = 0;

    printf("\n--- OPTIMAL ---\n");
    for (int i = 0; i < PAGES; i++)
    {
        if (!inFrames(frames, pages[i])) {
            if (filled < FRAMES) {
                frames[filled++] = pages[i];
            } else {
                // Find the frame whose page is used farthest in
                future int replace = 0, farthest = -1;
```

```
for (int j = 0; j < FRAMES; j++) {
```

```
                    int nextUse = PAGES; // assume never used again
                    for (int k = i + 1; k < PAGES; k++) {
                        if (pages[k] == frames[j]) { nextUse = k; break; }
                    }
                    if (nextUse > farthest) { farthest = nextUse; replace = j;
}
                }
                frames[replace] = pages[i];
            }
            faults++;
            printf("Page %d -> [%d %d %d] FAULT\n", pages[i], frames[0],
frames[1], frames[2]);
        } else {
            printf("Page %d -> [%d %d %d] HIT\n", pages[i], frames[0],
frames[1], frames[2]);
        }
    }
    printf("Total Faults: %d\n", faults);
}

int main() {
    printf("Page String: 7 0 1 2 0 3 0 4 2 3 0 3");
    printf("\nFrames: %d", FRAMES);
    FIFO();
    LRU();
    Optimal();
    return 0;
}
```

## OUTPUT


C:\Users\BMSCECSE-L4\Deskt
Page String: 7 0 1 2 0 3 0 4 2 3 0 3
Frames: 3
--- FIFO ---
Page 7 -> [7 -1 -1] FAULT
Page 0 -> [7 0 -1] FAULT
Page 1 -> [7 0 1] FAULT
Page 2 -> [2 0 1] FAULT
Page 0 -> [2 0 1] HIT
Page 3 -> [2 3 1] FAULT
Page 0 -> [2 3 0] FAULT
Page 4 -> [4 3 0] FAULT
Page 2 -> [4 2 0] FAULT
Page 3 -> [4 2 3] FAULT
Page 0 -> [0 2 3] FAULT
Page 3 -> [0 2 3] HIT
Total Faults: 10
--- LRU ---
Page 7 -> [7 -1 -1] FAULT
Page 0 -> [7 0 -1] FAULT
Page 1 -> [7 0 1] FAULT
Page 2 -> [2 0 1] FAULT
Page 0 -> [2 0 1] HIT
Page 3 -> [2 0 3] FAULT
Page 0 -> [2 0 3] HIT
Page 4 -> [4 0 3] FAULT
Page 2 -> [4 0 2] FAULT
Page 3 -> [4 3 2] FAULT
Page 0 -> [0 3 2] FAULT
Page 3 -> [0 3 2] HIT
Total Faults: 9
--- OPTIMAL ---
Page 7 -> [7 -1 -1] FAULT
Page 0 -> [7 0 -1] FAULT
Page 1 -> [7 0 1] FAULT
Page 2 -> [2 0 1] FAULT
Page 0 -> [2 0 1] HIT
Page 3 -> [2 0 3] FAULT
Page 0 -> [2 0 3] HIT
Page 4 -> [2 4 3] FAULT
Page 2 -> [2 4 3] HIT
Page 3 -> [2 4 3] HIT
Page 0 -> [0 4 3] FAULT
Page 3 -> [0 4 3] HIT
Total Faults: 7
Process returned 0 (0x0) execution time : 0.007 s
Press any key to continue.